

# TOPA 3급 온라인

## 01 파이썬 데이터 분석



# 목차

—

데이터 분석을 위한  
파이썬과 Numpy의 주요  
문법을 알아봅니다.

01 파이썬 자료형 (1)

02 파이썬 자료형 (2)

03 파이썬 조건문

04 파이썬 반복문



# 목차

—  
데이터 분석을 위한  
파이썬과 Numpy의 주요  
문법을 알아봅니다.

05 파이썬 제어문

06 파이썬 함수

07 Numpy 행렬

08 Numpy 연산



01

# 파이썬 자료형 (1)

## ✓ 변수란?

- 값을 저장하기 위해 할당하는 공간

|          |          |           |
|----------|----------|-----------|
| <b>x</b> | <b>=</b> | <b>10</b> |
| 변수 이름    |          | 값         |

→ 변수 x에 10이라는 값을 할당한다.

|          |           |           |
|----------|-----------|-----------|
| <b>x</b> | <b>=</b>  | <b>10</b> |
| <b>y</b> | <b>=</b>  | <b>10</b> |
| <b>x</b> | <b>==</b> | <b>y</b>  |

→ 변수 x와 변수 y의 값이 같다.

### ✓ 자료형이란?

- 프로그래밍 언어에서 사용되는 **변수의 데이터 타입**

$x = 10 \rightarrow x$ 는 정수형

$x = 2.5 \rightarrow x$ 는 실수형

$x = \text{"Hello World"} \rightarrow$  문자열

## ④ 자료형의 종류

|      |                                               |                               |
|------|-----------------------------------------------|-------------------------------|
| 수치형  | int                                           | -10, -5, 0, 1, 10, 30, 100    |
|      | float                                         | 0.1, -0.3, 0.005, 1e-10, 3e-7 |
|      | complex                                       | 2+3j, 2-3j, 3-4j, 3+4j        |
| 논리형  | True, False                                   |                               |
| 문자열  | "Hello World", "나는야 코딩천재", "123", "0.123"     |                               |
| 리스트  | [1, 3, 5], ['a', 'b', 'c'], [123, '123', 'a'] |                               |
| 튜플   | (1, 3, 5), ('a', 'b', 'c'), (123, '123', 'a') |                               |
| 딕셔너리 | { '국어': 95, '수학': 88, '영어': 55 }              |                               |
| 집합   | {Kim, Choi, Chang, Lee}                       |                               |

## ✓ int 자료형

- 정수형을 나타내는 자료형
- 범위:  $-\infty \sim +\infty$

```
a = 30
b = 2
c = 0
d = -12

print(a)
print(b)
print(c)
print(d)
print(type(a))
```

### Result

```
30
2
0
-12
<class 'int'>
```



## ☑ float 자료형

- 실수형을 나타내는 자료형
- 범위:  $4.9 \times 10^{-324} \sim 1.8 \times 10^{308}$

```
a = 3.4
b = -2.5
c = 3.5e-3
d = 2e+3

print(a)
print(b)
print(c)
print(d)
print(type(a))
```

### Result

```
3.4
-2.5
0.0035
2000.0
<class 'float'>
```

## ☑ int형과 float형 사이의 형변환

- float to int → `int(변수)`
- int to float → `float(변수)`

```
#float to int  
a = 3.7  
b = int(a)  
print('float to int')  
print(a)  
print(b)
```

```
#int to float  
a = 3  
b = float(a)  
print('int to float')  
print(a)  
print(b)
```

### Result

```
float to int  
3.7  
3  
  
int to float  
3  
3.0
```

## ☑ 수치형 연산

```
a = 8
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a ** b)
print(a % b)
print(a // b)
```

### Result

```
11
5
24
2.6666666666666665
512
2
2
```

## ☑ 수치형 연산

```
a, b, c, d, e, f, g = 8, 3, 2, 6, 11, 3, 5
```

```
a -= 3 # a = a - 3
```

```
b += 2 # b = b + 2
```

```
c *= 2 # c = c * 2
```

```
d /= 5 # d = d / 5
```

```
e //= 2 # e = e // 2
```

```
f **= 2 # f = f ** 2
```

```
g %= 2 # g = g % 2
```

```
print('a -= 3 →', a)
```

```
print('b += 2 →', b)
```

```
print('c *= 2 →', c)
```

```
print('d /= 5 →', d)
```

```
print('e //= 2 →', e)
```

### Result

a -= 3 → 5

b += 2 → 5

c \*= 2 → 4

d /= 5 → 1.2

e //= 2 → 5

f \*\*= 2 → 9

g %= 2 → 1



### ☑ string 자료형

- 문자, 단어, 문장 등을 나타내는 자료형
- " " 또는 ' ' 안에 문자열을 넣어 선언

```
a = "Hello World"  
b = 'Hello World'  
  
print(a)  
print(b)  
print(type(a))
```

#### Result

```
Hello World  
Hello World  
<class 'str'>
```

## ☑ string 자료형

영희가 말했어요 "철수야 밥 먹었니?"

```
a = "영희가 말했어요 "철수야 밥 먹었니?""  
b = "영희가 말했어요 \"철수야 밥 먹었니?\""  
c = '영희가 말했어요 "철수야 밥 먹었니?"'
```

→ 잘못된 문자열 선언

## ☑ string 자료형

- **+ 와 \* 연산자를 활용**하여 문자열 생성 가능

```
print(1 + 1)
print('a' + 'b')

a = "Hello " * 4
print(a)
```

### Result

2

ab

Hello Hello Hello Hello

## ☑ string 자료형

- **Indexing**: 문자열의 특정 위치에 있는 문자 반환

| 문자열 | H   | e   | l  | l  | o  |    | W  | o  | r  | l  | d  |
|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| 인덱스 | 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|     | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

```
a = "Hello World"
print(a[0])
print(a[3])
print(a[5])
print(a[-1])
print(a[-3])
print(a[-5])
```

### Result

```
H
l

d
r
W
```

## ☑ string 자료형

- **Slicing**: 전체 문자열에서 특정 부분에 해당하는 문자열 반환  
- [start:end:stride] 형태로 사용

| 문자열 | H   | e   | l  | l  | o  |    | W  | o  | r  | l  | d  |
|-----|-----|-----|----|----|----|----|----|----|----|----|----|
| 인덱스 | 0   | 1   | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|     | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |

```

a = "Hello World"
print(a[0:3:1], a[0:3], a[:3])
print(a[-3:])
print(a[::-1], a[::-2])
print(a[:])

```

### Result

```

Hel Hel Hel
rld
dlroW olleH drWoLH
Hello World

```



### ☑ 주요 문자열 메서드

- count(): 문자열 개수 반환
- replace(): 특정 문자열 치환
- split(): 특정 문자열로 분리하여 반환
- join(): 문자열 리스트를 결합
- find(): 해당 문자열 위치 반환  
(없다면 -1 반환)
- index(): 해당 문자열 위치 반환  
(없다면 'value error' 반환)
- upper(): 대문자로 변환
- lower(): 소문자로 변환

## ☑ 주요 문자열 메서드

```
a = "apple"
print(a.count('p'))
print(a.find('p'))
print(a.index('p'))
print('.'.join(a))
a = a.upper()
print(a)
print(a.lower())
```

### Result

```
2
1
1
a.p.p.l.e
APPLE
apple
```

## ☑ 주요 문자열 메서드

```
b = "How can I improve my coding skills?"  
print(b)  
  
b = b.replace('?', '!')  
print(b)  
  
word_list = b.split(' ')  
print(word_list)
```

### Result

```
How can I improve my coding skills?  
How can I improve my coding skills!  
['How', 'can', 'I', 'improve', 'my', 'coding', 'skills!']
```

02

## 파이썬 자료형 (2)



### ✓ list 자료형

- **순서가 있는 데이터들의 집합**을 나타내는 데이터 타입
- 대괄호([])를 사용하여 선언

```
a = ["Hello", "World"]
b = [1, 2, [3, 4]]
c = ["Hello", 1, '2', True]
print(type(a))
print(a)
print(b)
print(c)
```

#### Result

```
<class 'list'>
['Hello', 'World']
[1, 2, [3, 4]]
['Hello', 1, '2', True]
```



## ✓ list 자료형

| 리스트 | Hello | World | !  | I  | like | python | !  |
|-----|-------|-------|----|----|------|--------|----|
| 인덱스 | 0     | 1     | 2  | 3  | 4    | 5      | 6  |
|     | -7    | -6    | -5 | -4 | -3   | -2     | -1 |

```
a = ["Hello", "World", "!", "I", "like", "python", "!"]
print(a[0])
print(a[-1])
a[1] = "Everyone"
print(a)
```

## Result

```
Hello
!
['Hello', 'Everyone', '!', 'I', 'like', 'python', '!']
```

## ④ list 자료형

| 리스트 | Hello | World | !  | I  | like | python | !  |
|-----|-------|-------|----|----|------|--------|----|
| 인덱스 | 0     | 1     | 2  | 3  | 4    | 5      | 6  |
|     | -7    | -6    | -5 | -4 | -3   | -2     | -1 |

```
a = ["Hello", "World", "!", "I", "like", "python", "!"]  
print(a[:5])  
print(a[2:5:2])  
print(a[1:-3])  
print(a[::-1])
```

## Result

```
['Hello', 'World', '!', 'I', 'like']  
['!', 'like']  
['World', '!', 'I']  
['!', 'python', 'like', 'I', '!', 'World', 'Hello']
```

### ☑ list 주요 메서드

- `append()`: 가장 마지막에 데이터 추가
- `sort()`: 오름차순 정렬
- `reverse()`: 순서 뒤집기
- `insert()`: 특정 위치에 데이터 추가
- `remove()`: 해당되는 데이터 값을 삭제
- `pop()`: 가장 마지막 값을 반환하고 그 값 삭제
- `extend()`: 가장 마지막에 데이터 추가  
(데이터 내부 원소를 추가)

### ✓ list 주요 메서드

```
#sort
a = ["Hello", "World", "Python"]
a.sort()
print("sort result: ", a)
```

#### Result

```
sort result: ['Hello', 'Python', 'World']
```

```
#reverse
a = ["Hello", "World", "python"]
a.reverse()
print('reverse result:', a)
```

#### Result

```
reverse result: ['python', 'World', 'Hello']
```

```
#index
a = ["Hello", "World", "python"]
print('index result:', a.index('Hello'))
```

#### Result

```
index result: 0
```

### ☑ list 주요 메서드

```
#append  
a = ["Hello", "World", "python"]  
a.append('good')  
print('append result:', a)
```

#### Result

```
append result: ['Hello', 'World', 'python', 'good']
```

```
a = ["Hello", "World", "python"]  
a.insert(2, 'index1')  
print(insert result: ', a)
```

#### Result

```
insert result: ['Hello', 'World', 'index1', 'python']
```



### ☑ list 주요 메서드

```
#extend  
a = ["Hello", "World", "python"]  
b = ["data", "science"]  
a.extend(b)  
print('extend result:', a)
```

#### Result

```
extend result: ['Hello', 'World', 'python', 'data', 'science']
```

```
#extend vs append  
a = ["Hello", "World", "python"]  
b = ["data", "science"]  
a.append(b)  
print('append result:', a)
```

#### Result

```
append result: ['Hello', 'World', 'python', ['data', 'science']]
```

### ✓ list 주요 메서드

```
#remove
a = ["Hello", "World", "python"]
a.remove('World')
print('remove result:', a)
```

#### Result

```
remove result: ['Hello', 'python']
```

```
#pop
a = ["Hello", "World", "python"]
print(a.pop())
print('pop result:', a)
```

#### Result

```
python
pop result: ['Hello', 'World']
```

```
#del
a = ["Hello", "World", "python"]
del a[2]
print('del result:', a)
```

#### Result

```
del result: ['Hello', 'World']
```

## ☑ tuple 자료형

- List와 같이 **순서가 있는 데이터들의 집합**이지만 **데이터 변경 불가능**
- 괄호()를 사용하여 선언 (괄호 생략 가능)

```
a = ("Hello", "World", "python")
b = (1, 2, 3)
c = 1, "hi", 2
print(type(a), type(b), type(c))
print(a)
print(b)
print(c)
```

### Result

```
<class 'tuple'> <class 'tuple'> <class 'tuple'>
('Hello', 'World', 'python')
(1, 2, 3)
(1, 'hi', 2)
```

## ☑ tuple 자료형

- List와 같이 **순서가 있는 데이터들의 집합**이지만 **데이터 변경 불가능**
- 괄호()를 사용하여 선언 (괄호 생략 가능)

```
l = [1, 2, 3]
l[2] = 1
print(l)
```

Result

```
[1, 2, 1]
```

```
t = (1, 2, 3)
t[2] = 1
print(t)
```

Result

```
TypeError: 'tuple' object does not support item assignment
```

## ④ tuple 자료형

| 튜플  | 1  | 2  | 3  | 4  |
|-----|----|----|----|----|
| 인덱스 | 0  | 1  | 2  | 3  |
|     | -4 | -3 | -2 | -1 |

```
a = 1, 2, 3, 4  
print(a[2], a[1:3])
```

**Result**

3 (2, 3)



### ☑ list형과 tuple형 사이의 형변환

```
a = ["Hello", "World", "!"]  
b = tuple(a)  
print(type(a), type(b))
```

**Result**

```
<class 'list'> <class 'tuple'>
```

```
a = ("Hello", "World", "!")  
b = list(a)  
print(type(a), type(b))
```

**Result**

```
<class 'tuple'> <class 'list'>
```

## ☑ Dictionary 자료형

- **key와 value의 쌍으로** 데이터가 순서 없이 모여 있는 데이터 타입
- 중괄호 {} 안에 'key : value' 형태로 선언 **{'key' : 'value'}**

```
a = {1: 'one',  
      'A': '가',  
      '과목': ['수학', '영어']}  
print(type(a))  
print(a)
```

### Result

```
<class 'dict'>  
{1: 'one', 'A': '가', '과목': ['수학', '영어']}
```

### ✓ Dictionary 주요 메서드

- keys(): key 반환
- values(): value 반환
- items(): key와 value 반환
- clear(): dictionary 데이터 모두 삭제

```
dic = {1: 'one',  
       'A': ['deep', 'learning'],  
       '숫자': 1234}  
print(dic.keys())  
print(dic.values())  
print(dic.items())  
dic.clear()  
print(dic)
```

#### Result

```
dict_keys([1, 'A', '숫자'])  
dict_values(['one', ['deep', 'learning'], 1234])  
dict_items([(1, 'one'), ('A', ['deep', 'learning']), ('숫자', 1234)])  
{}
```

### ☑ Dictionary 자료형

```
a = {1: "one",  
     "two": 2,  
     3: "three"}  
print(a)  
del a['two']  
print(a)
```

#### Result

```
{1: 'one', 'two': 2, 3: 'three'}  
{1: 'one', 3: 'three'}
```

### ✓ Set 자료형

- **중복 값을 허용하지 않는 집합 형태의** 데이터 타입
- 교집합, 차집합, 합집합과 같은 **집합의 연산 가능**
- 리스트 데이터에서 중복을 제거할 때 사용
- 딕셔너리와 같이 순서가 없는 데이터 타입

```
s = {1, 2, 3, 3, 4, 5, 5, 6, 7, 1, 2, 1}
print(type(s))
print(s)
```

#### Result

```
<class 'set'>
{1, 2, 3, 4, 5, 6, 7}
```

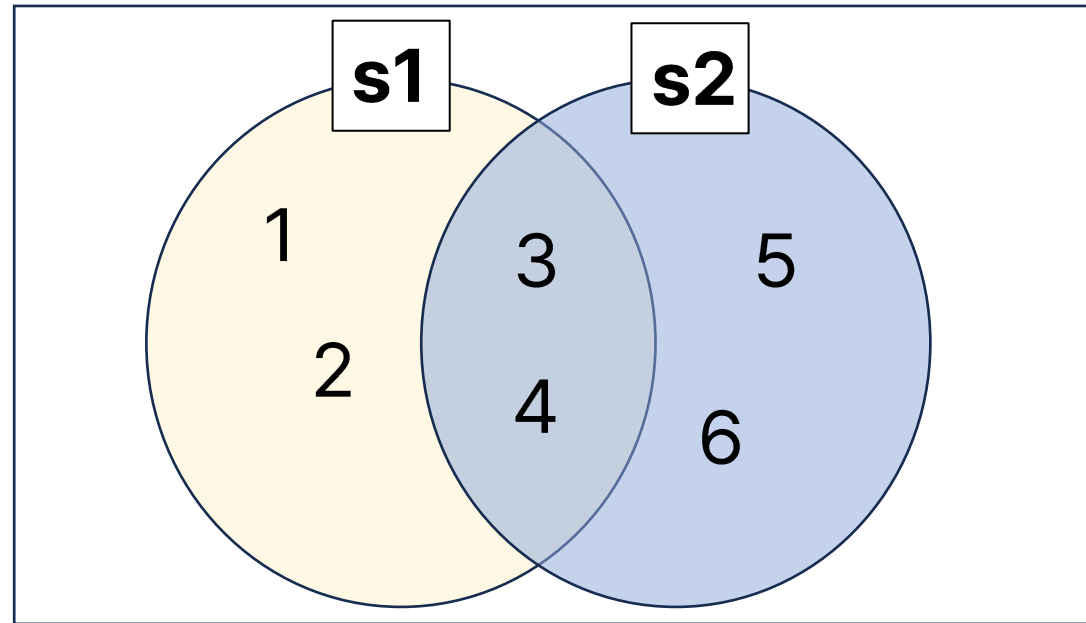
## ✓ Set 자료형

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1 & s2)
print(s1.intersection(s2))
```

**Result**

```
{3, 4}
{3, 4}
```

교집합 (intersection)

 $s1 = \{1, 2, 3, 4\}$  $s2 = \{3, 4, 5, 6\}$  $s1 \cap s2 = \{3, 4\}$



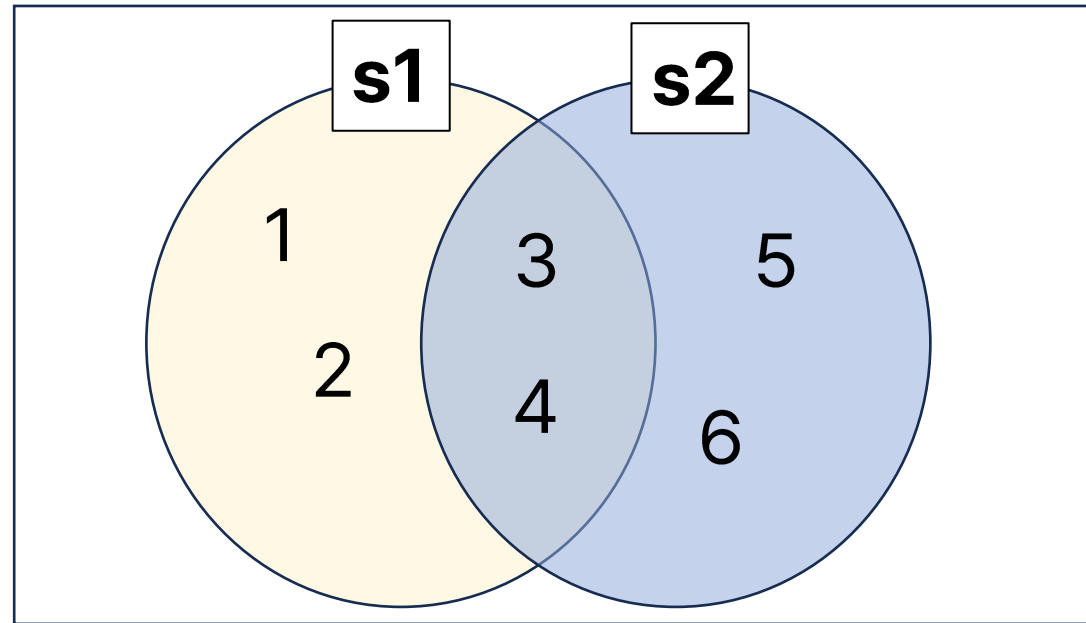
## ✔ Set 자료형

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1 | s2)
print(s1.union(s2))
```

## Result

```
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
```

합집합 (union)

 $s1 = \{1, 2, 3, 4\}$  $s2 = \{3, 4, 5, 6\}$  $s1 \cup s2 = \{1, 2, 3, 4, 5, 6\}$

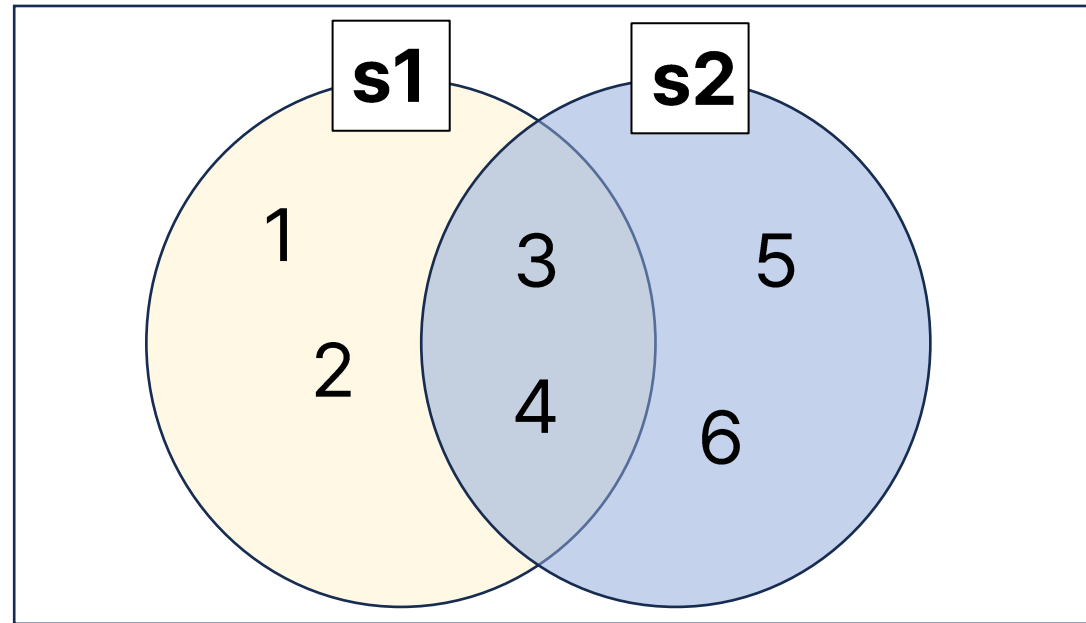
## ✓ Set 자료형

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
print(s1 - s2)
print(s1.difference(s2))
```

**Result**

```
{1, 2}
{1, 2}
```

차집합 (difference)

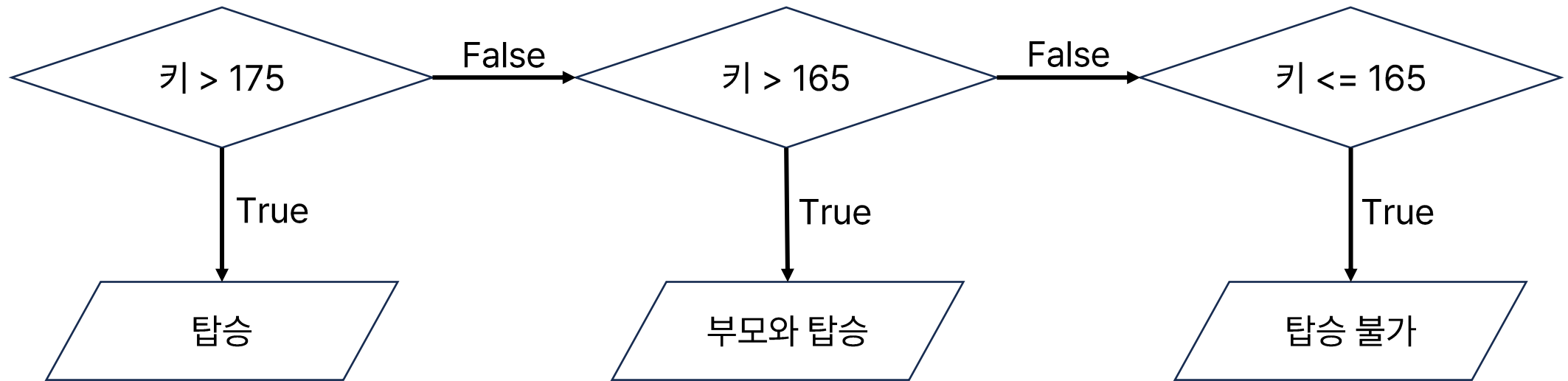
 $s1 = \{1, 2, 3, 4\}$  $s2 = \{3, 4, 5, 6\}$  $s1 - s2 = \{1, 2\}$

03

## 파이썬 조건문

## ✓ 조건문이란?

조건에 따라 실행되는 제어문



## ✓ 조건문이란?

```
height = 180
if height > 175:
    print('175cm 이상입니다. 탑승하세요.')
else:
    print('고객님은 탑승할 수 없습니다.')
```

#### Result

175cm 이상입니다. 탑승하세요.

```
height = 160
if height > 175:
    print('175cm 이상입니다. 탑승하세요.')
else:
    print('고객님은 탑승할 수 없습니다.')
```

#### Result

고객님은 탑승할 수 없습니다.



### ✓ 조건문이란?

```
height = 170
if height > 175:
    print('175cm 이상입니다. 탑승하세요.')
elif height > 165:
    print('부모님과 탑승하세요.')
else:
    print('고객님은 탑승할 수 없습니다.')
```

#### Result

부모님과 탑승하세요.



### ☑ 조건문이란?

```
height = 170
with_parent = 'yes'
if height > 175:
    print('175cm 이상입니다. 탑승하세요.')
elif height > 165:
    if with_parent == 'yes':
        print('부모님과 탑승하세요.')
    else:
        print('부모님과 함께 탑승해야 합니다.')
else:
    print('고객님은 탑승할 수 없습니다.')
```

#### Result

부모님과 탑승하세요.

```
height = 170
with_parent = 'no'
if height > 175:
    print('175cm 이상입니다. 탑승하세요.')
elif height > 165:
    if with_parent == 'yes':
        print('부모님과 탑승하세요.')
    else:
        print('부모님과 함께 탑승해야 합니다.')
else:
    print('고객님은 탑승할 수 없습니다.')
```

#### Result

부모님과 함께 탑승해야 합니다.

04

# 파이썬 반복문

### ✓ 반복문이란?

- 프로그램이 특정 코드를 반복하여 실행하도록 하는 제어문
- for문과 while문으로 이루어져 있음

### ✓ 반복문 – for문

```
range(0, 5, 1)
```

```
range(5)
```

range(start, end, stride)



## ④ 반복문 – for문

```
for i in range(5):  
    print(i)
```

**Result**

0  
1  
2  
3  
4

```
for i in range(5):  
    print("안녕하세요")
```

**Result**

안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요

range 함수를 활용하여 for문 선언

## ④ 반복문 – for문

```
#리스트 순회  
a = [1, 2, 3, 4, 5]  
for i in a:  
    print(i)
```

## Result

1  
2  
3  
4  
5

```
#딕셔너리 순회  
dic = {'수학': 90, '영어': 100}  
for i, j in dic.items():  
    print('과목:', i, '점수:', j )
```

## Result

과목: 수학 점수: 90  
과목: 영어 점수: 100



## ④ 반복문 – for문

```
#enumerate
```

```
l = ["a", "b", "c"]
```

```
for index, value in enumerate(l):
```

```
    print(index, value)
```

**Result**

0 a

1 b

2 c

enumerate 함수를 활용하여 **리스트의 인덱스와 값에 동시에 접근 가능**

## ☑ 반복문 – 중첩 for문

```
#중첩 for문
for i in range(4):
    for j in range(2):
        print('*')
    print(i)
```

**Result**

```
*
*
0
*
*
1
*
*
2
*
*
3
```

```
#중첩 for문
for i in range(4):
    for j in range(2):
        print('*', end='')
    print(i)
```

**Result**

```
**0
**1
**2
**3
```



### "end="

python의 print문은 기본적으로  
출력문의 마지막에 '\n'이 포함되어 있음  
'\n'을 원하지 않는다면 'end='을 통해  
원하는 문자를 출력문 마지막에 포함시  
킬 수 있음

## ④ 반복문 - while문

```
a = 3
while a <= 5:
    a += 1
    print(a)
```

## Result

4  
5  
6

반복 1) a = 3인 상태에 while문 진입  
→ a += 1 코드로 인해 a = 4  
→ print(a) 코드에서 a 출력

반복 2) a = 4인 상태에서 while문 진입  
→ a += 1 코드로 인해 a = 5  
→ print(a) 코드에서 a 출력

반복 3) a = 5인 상태에서 while문 진입  
→ a += 1 코드로 인해 a = 6  
→ print(a) 코드에서 a 출력

반복 4) a = 6인 상태에서 while문 진입하려 했지만 while문의 조건이 a <= 5이므로 진입하지 못하고 while문 탈출

05

## 파이썬 제어문



## ☑ for문 - break

```
for i in range(1, 10):  
    if i%2 == 0:  
        print(i)
```

Result

2  
4  
6  
8

```
a = ["가", "나", "다", "라", "메롱", "마", "바", "사"]  
for i in a:  
    if i == "메롱":  
        print("메롱?")  
        break  
    print(i)
```

Result

가  
나  
다  
라  
메롱?

[break]

if문 조건에 해당되면 반복문 탈출



## ☑ while문 - break

while문의 조건이 True인 경우 무한번 반복  
if ~ break 문을 사용하여 반복을 멈춰야함

```
a = 3
while True:
    a -= 1
    if a == 0:
        break
    print(a)
```

Result

2  
1

[break]

if문 조건에 해당되면 반복문 탈출

- 반복 1) a = 3인 상태에 while문 진입  
→ a -= 1 코드로 인해 a = 2  
→ a == 0에 대한 조건에 해당하지 않으므로 if문 통과  
→ print(a) 코드에서 a 출력
- 반복 2) a = 2인 상태에서 while문 진입  
→ a -= 1 코드로 인해 a = 1  
→ a == 0에 대한 조건에 해당하지 않으므로 if문 통과  
→ print(a) 코드에서 a 출력
- 반복 3) a = 1인 상태에서 while문 진입  
→ a -= 1 코드로 인해 a = 0  
→ a == 0에 대한 조건에 해당하므로 break문 실행, 반복문 탈출

06

# 파이썬 함수

## ☑ 함수(Function)이란?

- 특정 작업을 수행하기 위해 독립적으로 설계된 코드의 집합

```
def data_science():  
    print("Hello World")  
  
data_science()
```

Result

Hello World

## ✓ 리턴 (return)

- 함수를 종료하며 결과 데이터 반환하는 용도로 사용

```
def no_return():  
    a = 'hi'
```

```
re = no_return()  
print(re)
```

**Result**

None

```
def do_return():  
    return 'hi'
```

```
re = do_return()  
print(re)
```

**Result**

hi

## ☑ 입력 값이 있는 함수

- **파라미터 (parameter)**: 함수 호출 시 전달하는 데이터를 받아서 함수 내에서 사용되는 변수

\* **sentence**: 파라미터

```
def data_science(sentence):  
    print(sentence)  
  
data_science("매개변수로 전달된 문자열 출력")
```

### Result

매개변수로 전달된 문자열 출력



## ☑ 디폴트 파라미터 (Default Parameter)

- 함수 호출시 파라미터에 대한 **입력 값이 없으면 디폴트로 설정된 값이 파라미터 변수로 입력됨**

```
def minus(num1=3, num2=2):  
    print(num1-num2)  
minus(2)
```

Result

0

num1: 입력 값 2, num2: default parameter 값 2

## ☑ 전역 변수와 지역 변수

- 전역(global) 변수: 코드 전체에서 사용 가능
  - 특정 블록 밖에서 선언되어 어디에서든 사용 가능한 변수
- 지역(local) 변수: 특정 블록에서만 사용 가능
  - 특정 블록 내에서 선언되어 그 블록 내에서만 사용 가능

print\_lv() 함수 내에서  
지역 변수 선언

```
def print_lv():  
    lv = 100  
    print(lv)  
  
print_lv()  
print(lv)
```

### Result

```
100  
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-77-9eee113b7eee> in <cell line: 6>()  
      4  
      5 print_lv()  
----> 6 print(lv)  
  
NameError: name 'lv' is not defined
```

print\_lv() 함수 내에서 지역 변수로 선언된 lv를  
함수 밖에서 사용했을 때 오류 발생

## ☑ 전역 변수와 지역 변수

- 전역(global) 변수: 코드 전체에서 사용 가능
  - 특정 블록 밖에서 선언되어 어디에서든 사용 가능한 변수
- 지역(local) 변수: 특정 블록에서만 사용 가능
  - 특정 블록 내에서 선언되어 그 블록 내에서만 사용 가능

전역 변수 gv 선언  
print\_gv() 함수 내에  
같은 이름의 지역 변수 gv 선언

print\_gv() 함수를 호출  
지역 변수 gv 출력  
전역 변수 gv 출력

```
gv = 10
def print_gv():
    gv = 100
    print(gv)

print_gv()
print(gv)
```

Result

100  
10

07

# Numpy 행렬



### ✓ 파이썬 라이브러리

- 개발자들에 의해서 만들어진 클래스나 함수의 모음집
- 필요할 때 언제든지 호출하여 사용할 수 있음
- 호출할 때는 import 함수를 사용하여 호출

```
import numpy as np
```



### as 명령어

- 패키지의 함수를 사용할 때마다 패키지의 전체 이름을 입력하는 수고스러움을 줄이기 위해 as 명령어를 사용함
- as 뒤에는 사용자가 원하는 패키지명의 줄임말 입력 가능

`numpy.array()` -> `np.array()`



## ☑ Numpy란?

- 컴퓨터 과학을 위한 기본적인 파이썬 패키지
- 다차원의 행렬 데이터 빠르게 처리 가능



#파이썬 리스트

```
a = [1, 2, 3]
b = [4, 5, 6]
print(a + b)
print(a * 5)
```

Result

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

VS

#넘파이 어레이  
import numpy as np

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)
print(a * 5)
```

Result

```
[5 7 9]
[ 5 10 15]
```

### ☑ 1차원 numpy 행렬 생성

```
import numpy as np

arr = np.array([1, 2, 3])
arr2 = np.array(range(5))
print(type(arr))
print(type(arr2))
print(arr)
print(arr2)
```

#### Result

```
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
[1 2 3]
[0 1 2 3 4]
```

## ☑ 2차원, 3차원 numpy 행렬 생성

```
import numpy as np

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

print(arr)
```

**Result**

```
[[1 2 3]
 [4 5 6]]
```

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
                [[7, 8, 9], [10, 11, 12]]])

print(arr)
```

**Result**

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
 [10 11 12]]]
```

## ☑ 차원과 모양 확인

```
import numpy as np

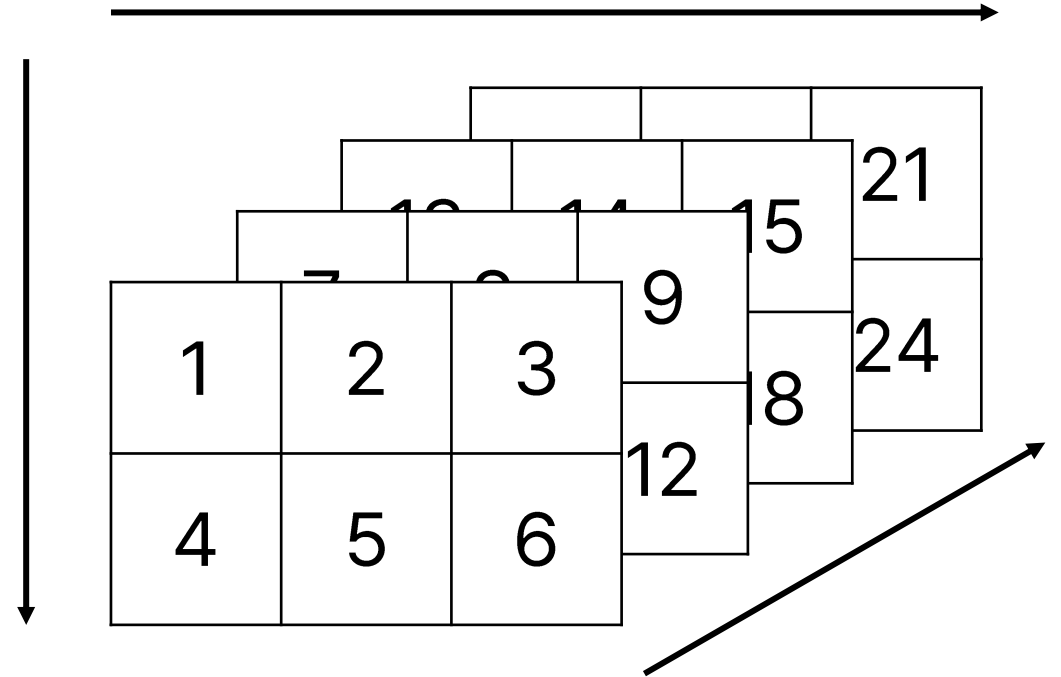
arr = np.array([[[1, 2, 3], [4, 5, 6]],
                [[7, 8, 9], [10, 11, 12]],
                [[13, 14, 15], [16, 17, 18]],
                [[19, 20, 21], [22, 23, 24]]])

print(arr.ndim)
print(arr.shape)
```

## Result

3  
(4, 2, 3)

→ 2행 3열짜리 데이터가 4개 있다는 의미



## ☑ range & arange를 통한 numpy 행렬 생성

```
import numpy as np

print(np.array(range(10)))
print(np.array(range(5,10)))
print(np.array(range(5,10,2)))
```

### Result

```
[0 1 2 3 4 5 6 7 8 9]
[5 6 7 8 9]
[5 7 9]
```

```
import numpy as np

print(np.arange(10))
print(np.arange(5,10))
print(np.arange(5,10,2))
```

### Result

```
[0 1 2 3 4 5 6 7 8 9]
[5 6 7 8 9]
[5 7 9]
```



## ☑ 모양 변경하기 (reshape)

```
import numpy as np

na = np.array(range(10))
print(na)
print(na.shape)
na = na.reshape(2,5)
print(na)
print(na.shape)
```

### Result

```
[0 1 2 3 4 5 6 7 8 9]
(10,)
[[0 1 2 3 4]
 [5 6 7 8 9]]
(2, 5)
```

## ☑ 모양 변경하기 (reshape)

```
import numpy as np

na = np.array(range(5))
print(na)
print(na.shape)
na = na.reshape(2,3)
print(na)
print(na.shape)
```

### Result

```
[0 1 2 3 4]
```

```
(5,)
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-89-909df5f0217d> in <cell line: 6>()
      4 print(na)
      5 print(na.shape)
----> 6 na = na.reshape(2,3)
      7 print(na)
      8 print(na.shape)
```

```
ValueError: cannot reshape array of size 5 into shape (2,3)
```

reshape 오류 예시

## ☑ 0-행렬 만들기 (zeros)

```
import numpy as np

zero = np.zeros((2, 3, 2), dtype=int)
print(zero)
```

### Result

```
[[[0 0]
  [0 0]
  [0 0]]
```

```
[[[0 0]
  [0 0]
  [0 0]]]
```



**'dtype ='**

자료형을 설정하는 옵션

### ✓ 1-행렬 만들기 (ones)

```
import numpy as np

one = np.ones((4, 3), dtype=int)
print(one)
```

#### Result

```
[[1 1 1]
 [1 1 1]
 [1 1 1]
 [1 1 1]]
```

## ☑ 균등한 확률로 정수 난수 발생 (randint)

```
import numpy as np

r = np.random.randint(5, 10, size=(2, 3))
print(r)
```

**Result**

```
[[6 5 8]
 [6 5 7]]
```

```
import numpy as np

r = np.random.randint(5, size=(4,3))
print(r)
```

**Result**

```
[[4 4 3]
 [3 1 3]
 [3 0 2]
 [0 1 0]]
```



## ☑ 행렬의 인덱싱

```
import numpy as np

arr = np.array(
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
)

print(arr[1])
print(arr[0][1])
print(arr[1][0][2])
```

|           |              |              |              |              |              |              |           |
|-----------|--------------|--------------|--------------|--------------|--------------|--------------|-----------|
|           | arr[0][0][0] | arr[0][0][1] | arr[0][0][2] | arr[1][0][0] | arr[1][0][1] | arr[1][0][2] |           |
| arr[0][0] | 1            | 2            | 3            | 7            | 8            | 9            | arr[1][0] |
| arr[0][1] | 4            | 5            | 6            | 10           | 11           | 12           | arr[1][1] |
|           | arr[0][1][0] | arr[0][1][1] | arr[0][1][2] | arr[1][1][0] | arr[1][1][1] | arr[1][1][2] |           |
|           | arr[0]       |              |              | arr[1]       |              |              |           |

## Result

```
[[ 7  8  9]
 [10 11 12]]
[ 4  5  6]
9
```

## ☑ 행렬의 슬라이싱

|           |              |              |              |              |              |              |           |
|-----------|--------------|--------------|--------------|--------------|--------------|--------------|-----------|
|           | arr[0][0][0] | arr[0][0][1] | arr[0][0][2] | arr[1][0][0] | arr[1][0][1] | arr[1][0][2] |           |
| arr[0][0] | 1            | 2            | 3            | 7            | 8            | 9            | arr[1][0] |
| arr[0][1] | 4            | 5            | 6            | 10           | 11           | 12           | arr[1][1] |
|           | arr[0][1][0] | arr[0][1][1] | arr[0][1][2] | arr[1][1][0] | arr[1][1][1] | arr[1][1][2] |           |
|           | arr[0]       |              |              | arr[1]       |              |              |           |

```
import numpy as np
```

```
arr = np.array(  
    [[1, 2, 3], [4, 5, 6]],  
    [[7, 8, 9], [10, 11, 12]]  
)
```

```
print(arr[1:, :, 1:])
```

## Result

```
[[[ 8  9]  
  [11 12]]]
```

## ☑ 행렬 데이터 수정

```
import numpy as np
```

```
ls = np.array(range(5))
```

```
print(ls)
```

```
ls[2] = 0
```

```
print(ls)
```

**Result**

```
[0 1 2 3 4]
[0 1 0 3 4]
```

```
import numpy as np
```

```
ls = np.array(range(5))
```

```
print(ls)
```

```
ls[3:] = 0
```

```
print(ls)
```

**Result**

```
[0 1 2 3 4]
[0 1 2 0 0]
```

|      | ls[0] | ls[1] | ls[2] | ls[3] | ls[4] |
|------|-------|-------|-------|-------|-------|
| 변경 전 | 0     | 1     | 2     | 3     | 4     |
| 변경 후 | 0     | 1     | 0     | 3     | 4     |

|      | ls[0] | ls[1] | ls[2] | ls[3] | ls[4] |
|------|-------|-------|-------|-------|-------|
| 변경 전 | 0     | 1     | 2     | 3     | 4     |
| 변경 후 | 0     | 1     | 2     | 0     | 0     |

08

# Numpy 연산

### ☑ 행렬의 비교 연산

```
import numpy as np

na = np.array([1, 2, 3])
na2 = np.array([4, 5, 6])
print(na == 2)
print(na2 > 4)
```

#### Result

```
[False  True False]
[False  True  True]
```



## ✓ 필터링 (filtering)

```
import numpy as np

na = np.array([1, 2, 3])
na2 = np.array([4, 5, 6])
print(na[na == 2])
print(na2[na2 > 4])
```

**Result**

```
[2]
[5 6]
```

```
import numpy as np

na3 = np.arange(10)
print(na3[na3%3 == 0])
```

**Result**

```
[0 3 6 9]
```

```
import numpy as np

na4 = np.array([1, 2, 3, 4, 5])
na5 = np.array([1, 0, 3, 0, 5])
print(na4[na4 == na5])
```

**Result**

```
[1 3 5]
```

조건에 해당하는 부분의 행렬 출력

## ☑ Numpy 메소드 (min, max, argmin, argmax)

```
import numpy as np

na = np.random.randint(1,10, size=(5))
print(na)
print(na.min())
print(na.max())
print(na.argmin())
print(na.argmax())
```

### Result

[5 7 2 3 8]

2

8

2

4

## ☑ Numpy 메소드 (sum)

```
import numpy as np

na = np.random.randint(10, size=(2,3))
print(na)
print()
print(np.sum(na))
print()
print(np.sum(na, axis=0))
print()
print(np.sum(na, axis=1))
```

### Result

```
[[7 8 0]
 [5 2 6]]
```

```
28
```

```
[12 10  6]
```

```
[15 13]
```

## ☑ Numpy 메소드 (mean, median, var, std)

```
import numpy as np

na = np.random.randint(10, size=(2,3))
print(na)
print()
print(np.mean(na))
print()
print(np.median(na))
print()
print(np.var(na))
print()
print(np.std(na))
```

### Result

```
[[5 0 2]
 [5 3 4]]
```

```
3.1666666666666665
```

```
3.5
```

```
3.1388888888888893
```

```
1.7716909687891083
```

## ☑ Numpy 메소드 (all, any)

```
import numpy as np

na1 = np.array([1, 2, 3])
na2 = np.array([1, 2, 3])
na3 = np.array([1, 0, 3])

print(np.all(na1 == na2))
print(np.all(na1 == na3))
print(np.any(na1 == na2))
print(np.any(na1 == na3))
```

### Result

```
True
False
True
True
```